Tutorials Exercises W3Schools app iOS & Android NEW Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

React `useCallback` Hook

‹ Previous Next ›

The `useCallback` Hook is used to memoize a callback function.

Memoizing a function means caching the result of a function so that it does not need to be recalculated.

The `useCallback` function only re-executes when one of its dependencies changes value.

This allows us to isolate resource intensive functions so that they will not automatically run on every render.

The `useCallback` and `useMemo` Hooks are similar:

`useMemo` returns a memoized *value*.

`useCallback` returns a memoized *function*.

Learn more about `useMemo` in the `useMemo` chapter.

W3Schools Tutorials Exercises
W3Schools app NEW iOS & Android
Sign In
CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

The `useCallback` Hook accepts two arguments.

```
useCallback(callback, dependencies)
```

`callback` : The function that you want to memoize.

`dependencies` : An array of dependencies for the callback function. The memoized callback will only change if one of these dependencies has changed.

Example:

Get your own React.js Server

Without `useCallback` :

```
//Without useCallback:
import React, { useState } from 'react';
import { createRoot } from 'react-dom/client';

// Child component that receives a function prop
const Button = React.memo(({ onClick, text }) => {
  alert(`Child ${text} button rendered`);
  return <button onClick={onClick}>{text}</button>;
});

// Parent component without useCallback
function WithoutCallbackExample() {
  const [count1, setCount1] = useState(0);
  const [count2, setCount2] = useState(0);

  // This function is recreated on every render
  const handleClick1 = () => {
    setCount1(count1 + 1);
  };

  const handleClick2 = () => {
    setCount2(count2 + 1);
```

```
    <div>
      <h2>Without useCallback:</h2>
      <p>Count 1: {count1}</p>
      <p>Count 2: {count2}</p>
      <Button onClick={handleClick1} text="Button 1" />
      <Button onClick={handleClick2} text="Button 2" />
    </div>
  );
}

createRoot(document.getElementById('root')).render(
  <WithoutCallbackExample />
);
```

Run Example »

REMOVE ADS

Try running the example above and click the buttons.

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

By using the `useCallback` hook, we can memoize the functions and only recreate them when their dependencies change.

When clicking Button 1, only Parent and Button 1 should re-render, and when clicking Button 2, only Parent and Button 2 should re-render:

Example:

With `useCallback`:

```
//With useCallback:
import React, { useState, useCallback } from 'react';
import { createRoot } from 'react-dom/client';

// Child component that receives a function prop
const Button = React.memo(({ onClick, text }) => {
  console.log(`${text} button rendered`);
  return <button onClick={onClick}>{text}</button>;
});

// Parent component with useCallback
function WithCallbackExample() {
  const [count1, setCount1] = useState(0);
  const [count2, setCount2] = useState(0);

  // These functions are memoized and only recreated when dependencies char
  const handleClick1 = useCallback(() => {
    setCount1(count1 + 1);
  }, [count1]);

  const handleClick2 = useCallback(() => {
    setCount2(count2 + 1);
  }, [count2]);

  console.log("Parent rendered");
```

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

```
      <p>Count 2: {count2}</p>
      <Button onClick={handleClick1} text="Button 1" />
      <Button onClick={handleClick2} text="Button 2" />
    </div>
  );
}

createRoot(document.getElementById('root')).render(
  <WithCallbackExample />
);
```

Run Example »

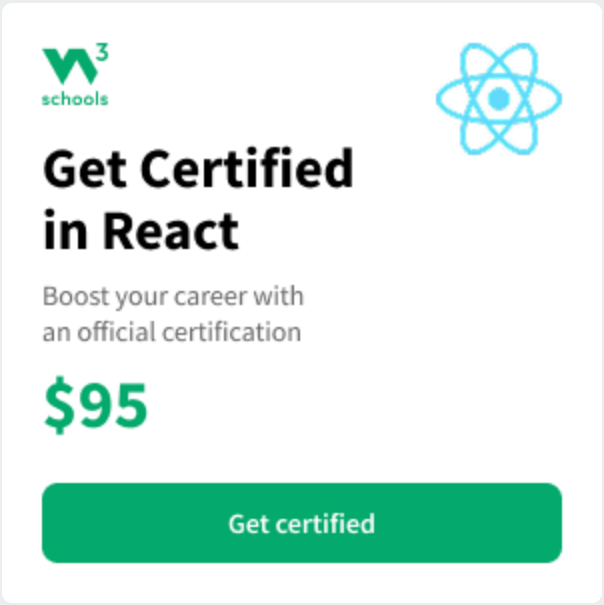
Exercise

What is the main purpose of useCallback?

- To store component state
- To memoize callback functions to prevent unnecessary re-renders
- To handle form submissions
- To manage component lifecycle

Submit Answer »

Tutorials Exercises
W3Schools app NEW iOS & Android
Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS



REMOVE ADS

W3Schools
Tutorials Exercises
W3Schools app NEW
iOS & Android
Sign In

. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

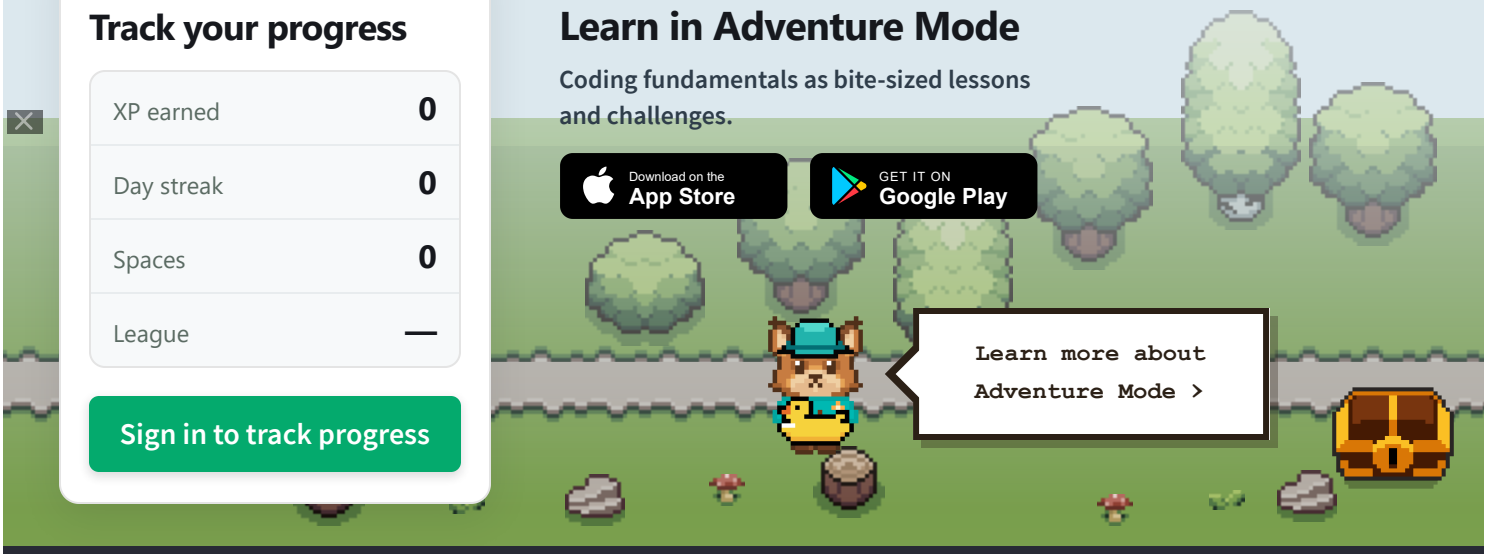
FREE ACCOUNT
Track your progress

XP earned	0
Day streak	0
Spaces	0
League	—

Sign in to track progress

NEW
Learn in Adventure Mode
Coding fundamentals as bite-sized lessons and challenges.

Download on the App Store
GET IT ON Google Play



W3Schools

PLUS
SPACES
GET CERTIFIED
FOR TEACHERS
PRACTICE
CONTACT US

Top Tutorials

HTML Tutorial
CSS Tutorial
JavaScript Tutorial
How To Tutorial
SQL Tutorial

W3Schools app NEW iOS & Android
Sign In

Tutorials Exercises

. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

jQuery Tutorial

Top References

HTML Reference
CSS Reference
JavaScript Reference
SQL Reference
Python Reference
W3.CSS Reference
Bootstrap Reference
PHP Reference
HTML Colors
Java Reference
AngularJS Reference
jQuery Reference

Top Examples

HTML Examples
CSS Examples
JavaScript Examples
How To Examples
SQL Examples
Python Examples
W3.CSS Examples
Bootstrap Examples
PHP Examples
Java Examples
XML Examples
jQuery Examples

Get Certified

HTML Certificate
CSS Certificate
JavaScript Certificate
Front End Certificate
SQL Certificate
Python Certificate
PHP Certificate
jQuery Certificate
Java Certificate
C++ Certificate
C# Certificate
XML Certificate

FORUM ABOUT CAMPUS

W3Schools is optimized for learning and training. Examples might be simplified to improve reading and learning.
Tutorials, references, and examples are constantly reviewed to avoid errors, but we cannot warrant full correctness
of all content. While using W3Schools, you agree to have read and accepted our terms of use, cookies and privacy policy.

Copyright 1999-2026 by Refsnes Data. All Rights Reserved. W3Schools is Powered by W3.CSS.